

CSE344 – System Programming

Homework #2

Signal Handling & Process Management with fork()

Due Date: 27 March 2026, 23:59

Platform: Debian 11 (64-bit) on VirtualBox

Grading: 50 pts Code + 50 pts Demo

Important Notes

- Identical or highly similar submissions will receive **-100 points**.
- A PDF report is **mandatory**. No report = 0 on the report section.
- The program must compile and run on **Debian 11 (64-bit)** in VirtualBox.
- Late submissions **will not be accepted**.
- A **demo** (~5 min) will be conducted for every student. The demo is worth **50 points** and is graded independently from the code.

Scenario

You will implement a multi-process file search tool in C using Linux system calls. The program accepts a root directory and a filename pattern, forks a configurable number of worker processes, and assigns each worker a partition of the directory tree to search recursively. Workers report results to the parent through signals, and the parent produces a final tree-formatted output with a summary. The program is named `procSearch` and must be built with a Makefile.

Usage

```
./procSearch -d <root_dir> -n <num_workers> -f <pattern> [-s <min_size_bytes>]
```

- **-d <root_dir>** Root directory to search (must exist)
- **-n <num_workers>** Number of worker processes to fork (between 2 and 8, inclusive)
- **-f <pattern>** Filename pattern; supports the `+` operator (see Section 1)
- **-s <min_size>** Optional. Match only files with size $\geq$ min_size bytes

If any required argument is missing or invalid, print usage information to `stderr` and exit with a non-zero code. Use `getopt()` for argument parsing.

1. Filename Pattern Matching (10 pts)

The `-f` pattern supports a limited regular expression: the `+` operator, which means one or more of the immediately preceding character. You must implement this from scratch — do not use any regex library.

```
Pattern: rep+ort      matches: report, repoort, repooort, ...
Pattern: lo+g         matches: log, loog, looog, ...
Pattern: er+ro+r      matches: error, erroorr, errorrr, ...
Pattern: notes        matches: notes (no + = exact match for that segment)
```

- Pattern matching is case-insensitive.
- Multiple `+` operators in a single pattern are allowed.
- The `+` operator applies only to the single character immediately before it.
- You may assume the user will never search for a literal `+` character.

2. Directory Partitioning (5 pts)

Before forking, the parent reads the **immediate subdirectories** of `<root_dir>` and distributes them among workers in round-robin order. Each worker is responsible for recursively searching its assigned subdirectories.

- If there are fewer immediate subdirectories than `num_workers`, reduce the worker count to match and print:

```
Notice: only 3 subdirectories found; using 3 workers instead of 5.
```

- If `<root_dir>` has no subdirectories at all, the parent searches it directly (no fork). Print:

```
Notice: no subdirectories found; parent will search root directly.
```

- The root directory itself is **not** assigned to any worker — only its immediate children are distributed.

3. Worker Processes — fork() (15 pts)

Each worker process, after being forked, must:

- Recursively traverse all assigned subdirectories.
- For every file that matches **both** the filename pattern and the optional size constraint, print to stdout in real time:

```
[Worker PID:4821] MATCH: /tmp/crawltest/alpha/report.txt (512 bytes)
```

- Track the total number of matches found and encode it in the exit status:
`exit(match_count % 256).`

- After finishing all assigned directories, send `SIGUSR1` to the parent process.
- If the worker receives `SIGTERM`, it must stop immediately, print its partial match count, and exit cleanly:

```
[Worker PID:4821] SIGTERM received. Partial matches: 3. Exiting.
```

4. Signal Handling (15 pts)

This is the core topic of the assignment. All four signals below must be correctly implemented.

SIGUSR1 — sent by Worker, received by Parent

Each time a worker finishes, it sends `SIGUSR1` to the parent. The parent counts how many workers have finished. When all workers have sent `SIGUSR1`, the parent calls `waitpid()` on each child to collect exit statuses and proceeds to print the final output.

The handler must be **async-signal-safe**. Use a `volatile sig_atomic_t` counter — do not call `printf()` inside the handler.

SIGINT (Ctrl+C) — received by Parent

The parent sends `SIGTERM` to all running workers, waits up to 3 seconds, then sends `SIGKILL` to any that remain. It then prints a shutdown message and exits.

Output: `[Parent] SIGINT received. Terminating all workers...`

After cleanup, print a partial summary of matches found so far.

SIGTERM — received by Worker

The worker stops crawling, prints its partial match count (see format above), and exits with the current match count as its exit status.

SIGCHLD — received by Parent

The parent must install a `SIGCHLD` handler to detect unexpected worker termination and prevent zombie processes. Call `waitpid(-1, &status, WNOHANG)` in a loop inside the handler.

If a worker exits unexpectedly (not due to `SIGTERM` from parent), log to `stderr`:

```
[Parent] Worker PID:4821 terminated unexpectedly (exit status: 1).
```

Workers are **not** automatically restarted.

5. Output Format (5 pts)

Workers print matching filenames in real time as they find them (see Section 3). After all workers finish, the parent prints a consolidated tree and summary.

Tree Format

```
<root_dir>
|-- alpha
|----- report.txt          (512 bytes)  [Worker 4821]
|----- sub1
|----- repoort.txt         (210 bytes)  [Worker 4821]
|-- beta
|----- report_final.txt     (88 bytes)   [Worker 4822]
|-- gamma
|----- repoooort.txt        (300 bytes)  [Worker 4823]
```

- Each subdirectory level is indented by 6 dashes (-----) relative to its parent.
- If no file matches: print **No matching files found**.
- All error messages go to stderr. Match output and the summary go to stdout.

Summary

```
--- Summary ---
Total workers used : 3
Total files scanned : 14
Total matches found : 4
Worker PID 4821    : 2 matches
Worker PID 4822    : 1 match
Worker PID 4823    : 1 match
```

Mandatory Test Scenario

You must test your program with the exact scenario below and include a screenshot of each step in your report. This sequence will be replicated during the demo.

1. Create the following directory tree under /tmp/crawltest:

```
/tmp/crawltest/
alpha/
  report.txt          (write: 'quarterly report data')
  notes.md
  sub1/
    repoort.txt       (write: 'old duplicate report')
    image.png         (empty)
beta/
  error_log.txt
  report_final.txt    (write: 'final version')
gamma/
  repoooort.txt       (write: 'archived report')
  data.csv
delta/
  summary.txt
```

2. Run: `./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort'` — verify that **report.txt**, **repoort.txt**, **repoooort.txt**, **report_final.txt** are all found and the tree output is correct.
3. Run again with `-s 15` — verify that only files with size ≥ 15 bytes appear in the results.

4. Test with a pattern that matches nothing (e.g., `-f 'xyz+123'`) — verify the output is `No matching files found.`
5. Run on a large real directory (e.g., `/usr/share/doc`) with `-n 4`. While it runs, send Ctrl+C and verify all workers terminate cleanly and no zombie processes remain (`ps -ef | grep procSearch`).
6. Show `monitor.log` (or `stderr` output) and verify SIGCHLD and SIGINT events are recorded.

Grading

Code — 50 Points

Component	Description	Points
Pattern Matching	Correct + operator, case-insensitive	10
Directory Partitioning	Round-robin split, edge cases handled	5
Worker Processes (fork)	Recursive search, match output, exit status	15
Signal Handling	SIGUSR1, SIGINT, SIGTERM, SIGCHLD all correct	15
Output & Summary	Tree format, summary accuracy	5
TOTAL		50

Makefile (`make` and `make clean`), code quality, and **-Wall** with 0 warnings are prerequisites for full credit. Missing or broken Makefile results in a penalty (see below).

Demo — 50 Points

Each student will attend an individual demo of approximately 5 minutes. The demo will be scheduled separately and conducted by the TA.

- You will run your program live on a test directory prepared by the TA.
- You will be asked questions about your implementation, design choices, and the system programming concepts involved.
- The demo is worth **50 points** and is graded independently from the code score.
- A student who cannot explain their own code will receive **0 on the demo** regardless of code quality.

Report Requirements (mandatory — no report = 0 on report)

1. Cover page: Name, Student ID, Course, Homework #
2. Introduction: what the program does and your overall design rationale

3. Design decisions: how you partitioned directories, how signal handlers are structured, how you avoid async-signal-safety issues

4. Screenshots: every step of the mandatory test scenario

5. Conclusion: challenges faced and how you resolved them

Tip: A report prepared with LaTeX earns +5 bonus points (submit .pdf and .tex together).

Penalties

Condition	Penalty
Code does not compile (make fails)	-100
fork() not used at all	-50
Zombie processes present (verified with ps)	-30
Non-async-signal-safe function (e.g., printf) called inside a signal handler	-20
volatile sig_atomic_t not used for handler flags	-15
Makefile missing or make clean does not work	-20
Each -Wall compiler warning (first 10)	-2 / warning
More than 10 -Wall warnings (no correction at demo)	-30
No report submitted, or report is clearly insufficient	-30
Binary executable files included in submission	-10
Late submission	Not accepted

Submission Rules

- Directory name: **your student number** (e.g., 210104066)
- Contents: all .c / .h source files, Makefile, report (**PDF only — other formats not accepted**)
- Archive: zip the directory → 210104066.zip
- Do **not** include compiled binaries. TAs compile on their own machines.
- The Makefile must **only compile** the program — it must not run it.
- Identical or suspiciously similar submissions will both receive **-100 points**.

Hints

- Plan carefully before writing any code. Draw the process tree and signal flow on paper first.
- Use sigaction() instead of signal() for reliable, portable signal handling.

- Signal handlers must only call async-signal-safe functions. Set a `volatile sig_atomic_t` flag in the handler and act on it in the main loop.
- Test for zombie processes frequently: `ps -ef | grep Z`
- Compile with `-Wall` early and often — fix warnings as they appear, not at the end.
- Test on at least two different directory structures before submitting.